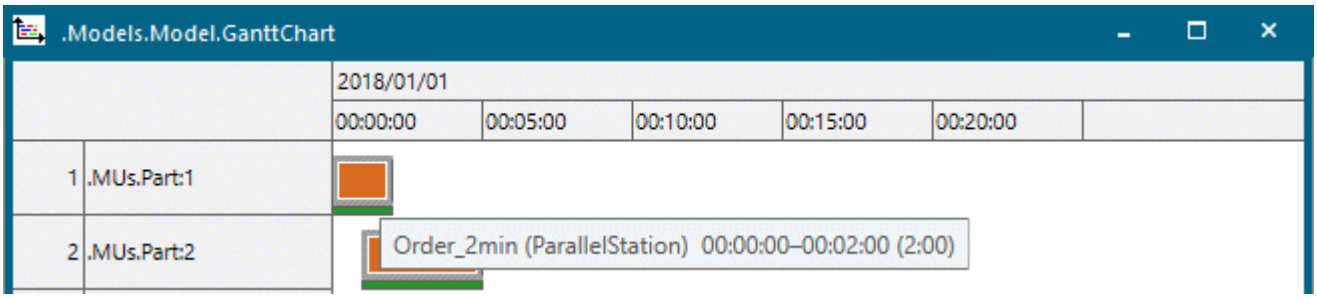
The displayed resource states refer to the resource which is currently occupied by the displayed part.



Show Longer or Shorter Time Intervals

To show **longer time** intervals, hold down **Ctrl** and press the **-** key or roll the mouse wheel backward. This zooms the view in.

To show **shorter time** intervals, hold down **Ctrl** and press the **+** key or roll the mouse wheel forward. This zooms the view out.

To change the displayed time intervals in **greater steps**, hold down **Shift** in addition to **Ctrl**.

To return to the **default zoom factor**, hold down **Ctrl** and press the 0 key.

Back to Show Parts on Resources in a GanttChart

Toggling States and Executing Actions

Switch States and Execute Actions

At times, you may want to toggle between the states *on* and *off*, or between different operating modes, etc. in your simulation model. Or you may want to execute an action by clicking a button in the model, for example open a certain table or method with one click instead of navigating to the object in the *Frame* and double-clicking it.

You can:

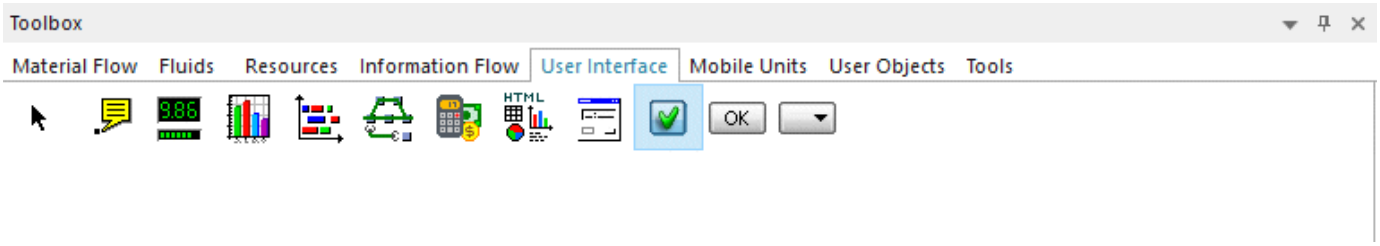
- Switch States with the Checkbox
- Execute the Action
- Select an Option from a Drop-down List

Back to Animate the Simulation Model and View the Results

Toggling States with the Checkbox

Switch States with the Checkbox

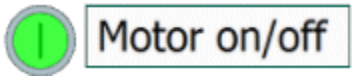
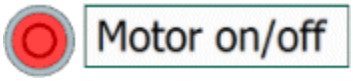
You can insert the **Checkbox** into your simulation model from the folder **UserInterface** in the *Class Library* or from the toolbar **User Interface** in the *Toolbox*.



To open the dialog of the object *Checkbox*, double-click the name **Checkbox** to the right of the icon in the *Frame* 

.

The *Checkbox* looks like this by default:

Appearance



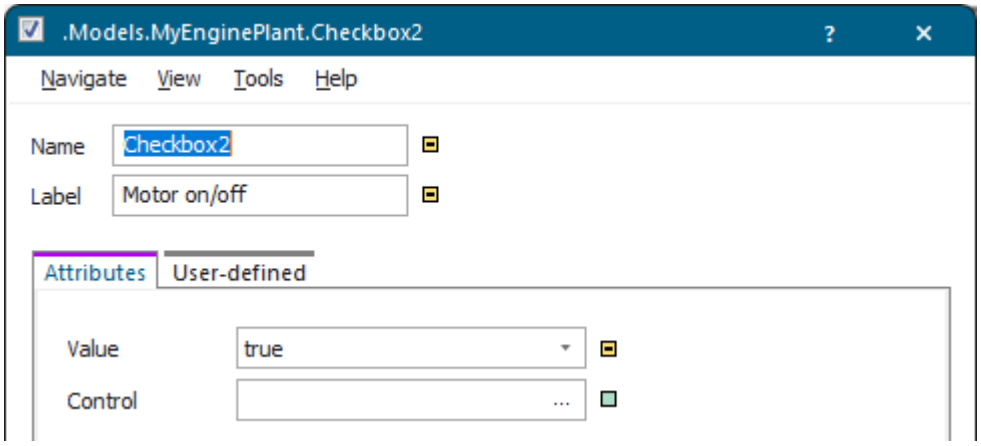
You can use the check box to:

- **Toggle the State by Clicking the Checkbox**
- **Switch Modes Using a Control**

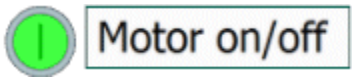
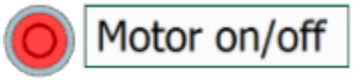
Back to Switch States and Execute Actions

Toggle the State by Clicking the Checkbox

To switch between the *on* and the *off* state of the *Checkbox*, click its icon in the *Frame*. To show an expression of your choice next to the check box, enter it into the text box **Label**. We entered `Motor on/off` in our example.



The *Checkbox* looks like this by default:

Appearance in 3D



When you click the check box in the *Frame*, it changes its icon from green, meaning *on*, to red, meaning *off*.

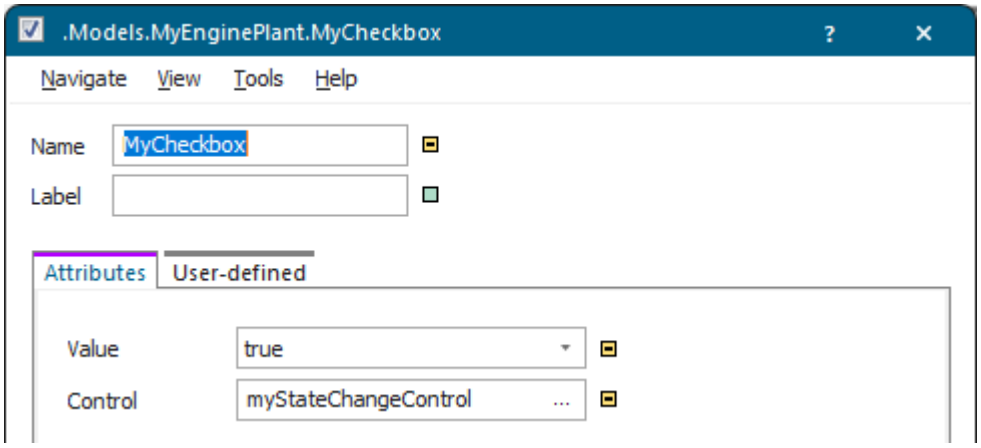
Back to Switch States with the Checkbox

Switch Modes Using a Control

If you want the check box to switch modes when a certain action takes place in your simulation model, you have to program this in a **Control**.

Double-click the name of the check box and select the control in which you programmed when the *Checkbox* switches modes.

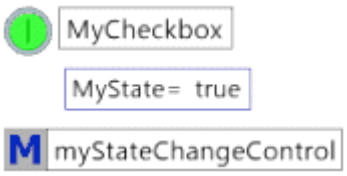
In our example the *Checkbox* named `MyCheckbox` changes the **Value** of the *Variable* named `MyState` from true to false and vice versa in the model.

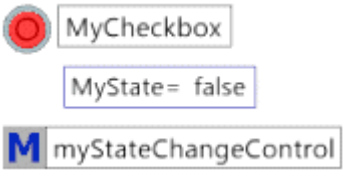


The source code looks like this:

```
MyState := MyCheckbox.Value
```

Clicking the *Checkbox* then switches the state.



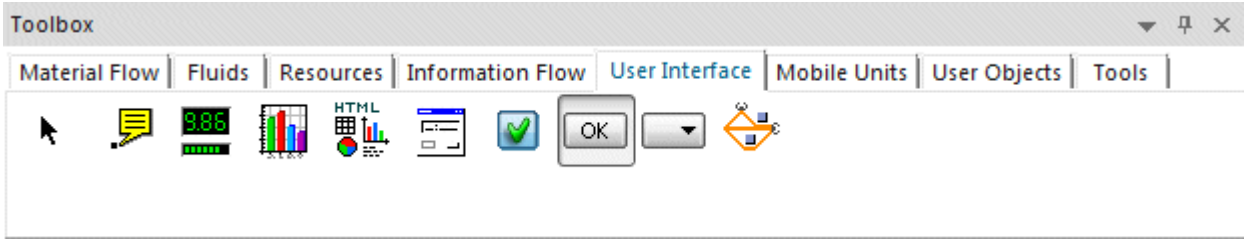


Back to Switch States with the Checkbox

Executing Actions by Clicking a Button

Execute the Action

You can insert the **Button** into your simulation model from the folder **UserInterface** in the *Class Library* or from the toolbar **User Interface** in the *Toolbox*.



We will demonstrate how to:

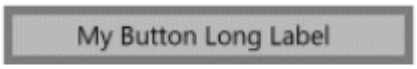
- **Define the Appearance of the Button**

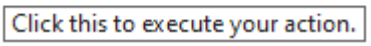
- **Program the Control for the Action to be Executed**

Back to Switch States and Execute Actions

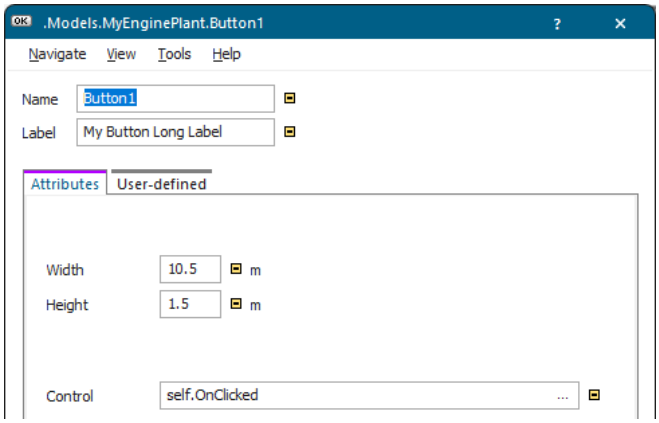
Define the Appearance of the Button

Plant Simulation shows the button in the *Frame* in a number of ways:

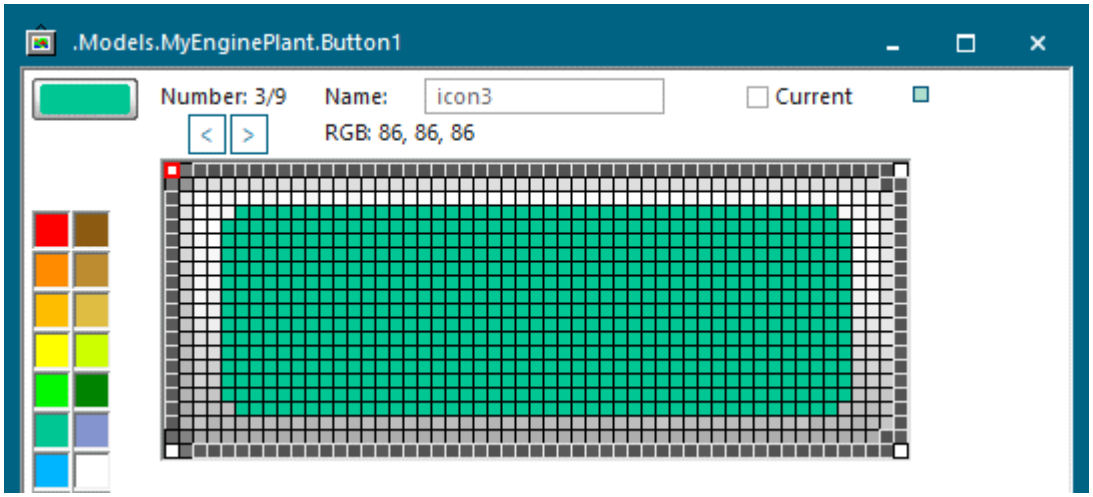


- With the built-in graphic and the **label** you type in, for example 

. You might have to adjust the **Width** and the **Height** so that the label fits the button without being cut off. You can do this by holding down **Ctrl+Shift** and dragging a corner of the icon. Or you can enter exact values into the text boxes. Or you can combine both methods by first dragging to roughly resize the button and then by fine-tuning the values in the text boxes.



- With a **user-defined icon**  , which you draw in the icon editor.



Then, you can show the text you enter as the **label** in this user-defined icon of the button, 

for example.

To be able to tell that you clicked the button, we recommend to draw two pictures for the button, one for the unclicked state 

, and one for the clicked state 

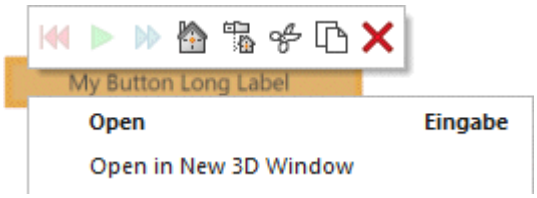
. In our example above the name of the unclicked icon is **icon3**, the name of the clicked icon consequently has to be **icon3_down**. The button does not stay pressed, but returns to its unclicked, raised state when you release the mouse button.

- With a **picture** you paste into a new icon in the *icon editor*,  and  for example.

Our example looks like this:

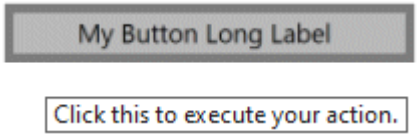
Built-in Icon, Not Clicked	Built-in Icon, Clicked
	

To open the dialog of the button, click it with the right mouse button and select **Open** on the context menu.

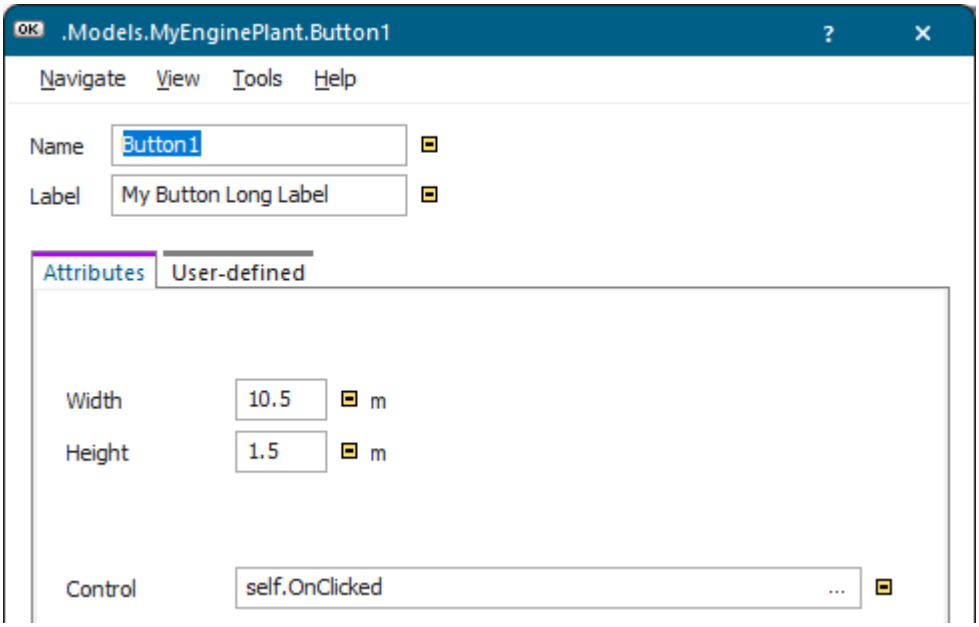


You can also select the button by dragging a marquee around it.

By default *Plant Simulation* does not show a tooltip for the button in the *Frame* when you roll the mouse over it. To display a your own *tooltip*, create a *user-defined attribute* and name it `Tooltip`.



After you have defined the appearance of the *Button*, you have to **program the action**, which the **Control** executes, when the *Button* is clicked.



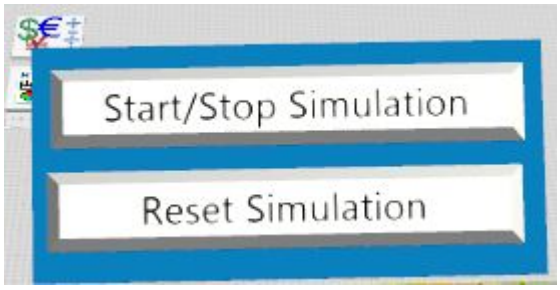
To execute this action, click the button in the *Frame*.

Back to Execute the Action

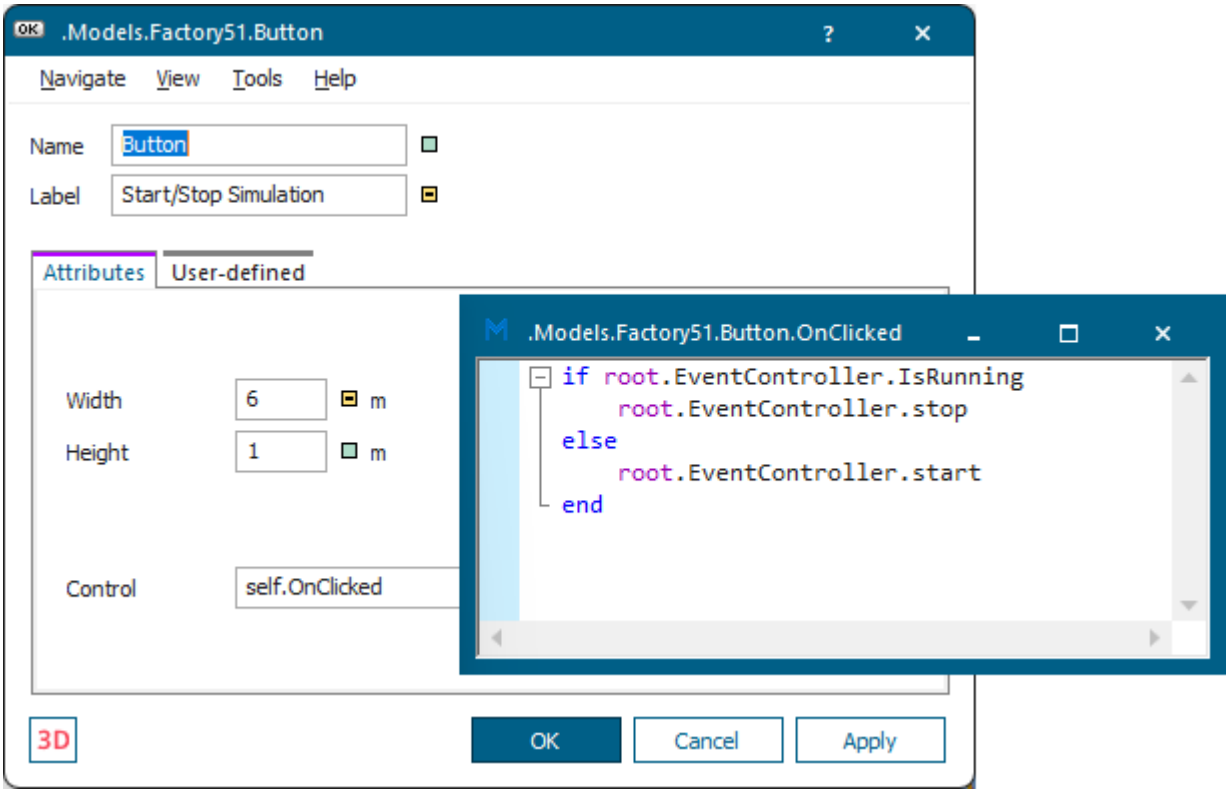
Program the Control for the Action to be Executed

After you defined the appearance of the *Button*, you have to program the control that clicking the *Button* executes.

We explain the control using the example of the **Start/Stop Simulation** button in our *Factory51* sample model.



We created our control by clicking the ellipsis button in the text box **Control** and selecting **Create Control** on the context menu.



We wrote this source code which checks if the *EventController* is running at the moment. If so, it stops the simulation, and then starts it.

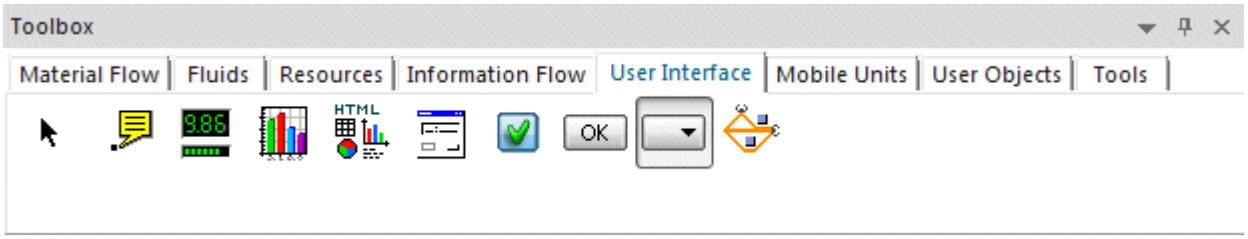
```
if root.EventController.IsRunning
...root.EventController.stop
else
...root.EventController.start
end
```

Back to Execute the Action

Selecting an Option from a Drop-down List

Select an Option from a Drop-down List

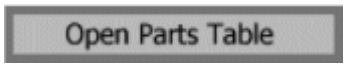
You can insert the **Drop-Down List** into your simulation model from the folder **UserInterface** in the *Class Library* or from the toolbar **User Interface** in the *Toolbox*.



In our sample model we show how to combine the objects *Button*, *Check Box*, and *Drop-down List* with user-defined controls to make meaningful use of them and how to access a number of settings in your simulation model quickly and easily.

- The *check box* 

activates or deactivates moving the parts on to a dedicated tray in the plant.
- The *drop-down list* 

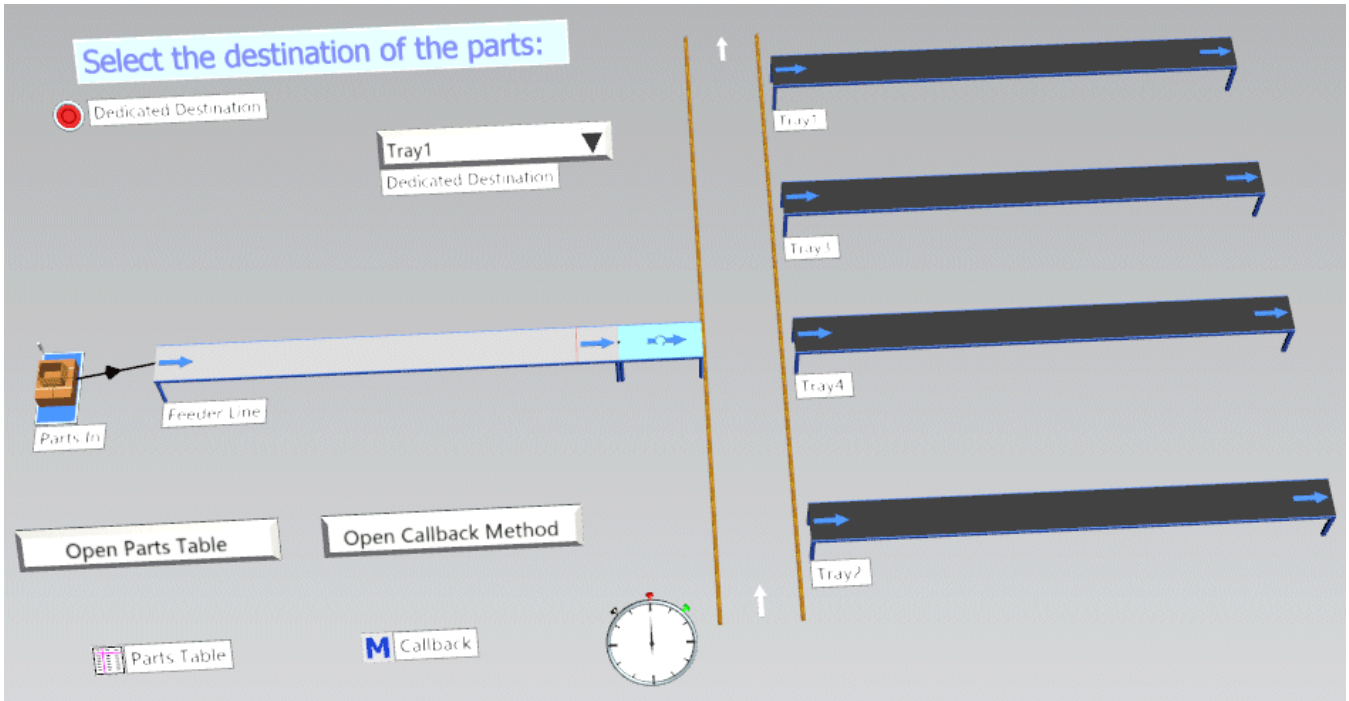
sets the dedicated destination tray of the parts if you turn the check box on.
- The *button* 

opens the table according to which the *Source* produces the parts.
- The *button* 

opens the sensor control of the feeder line that sends the parts on to their destination trays according to the settings of the check box 

and of the *drop-down list* 

.



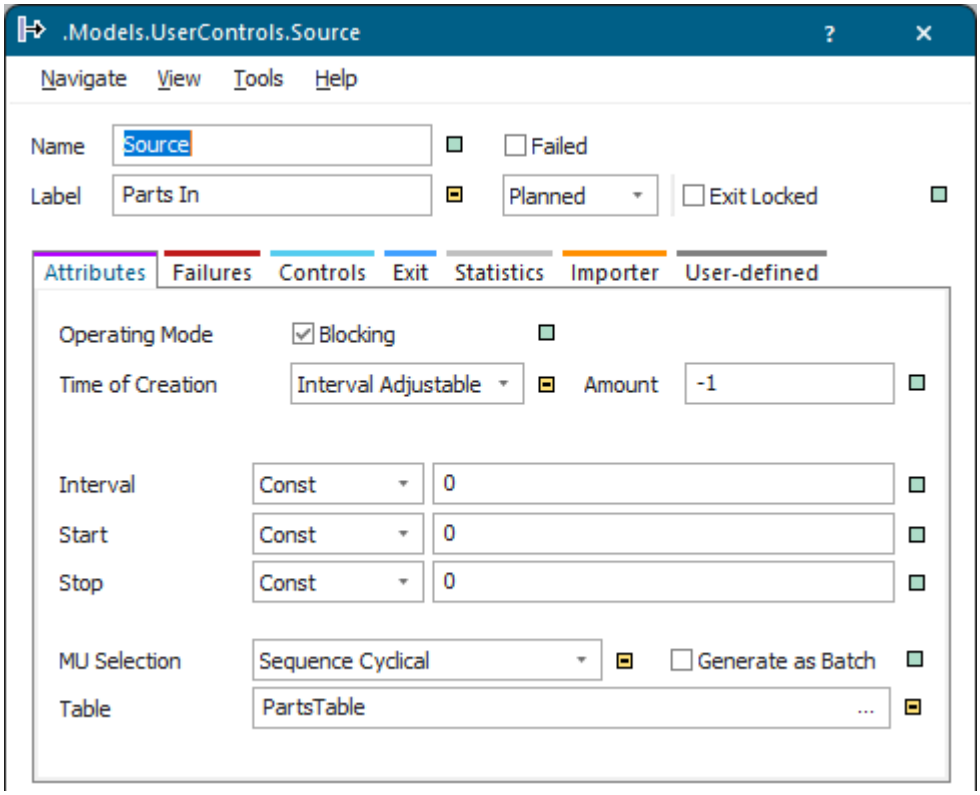
To create the sample model, we will:

- **Configure the Source that Produces the Parts [Drop-down List]**
- **Configure the Feeder Line with Sensor and Sensor Control**
- **Configure the Turnplate to Rotate the Part According to an Attribute**
- **Configure the Check Box and the Drop-down List**
- **Configure the Buttons to Open Parts Table and Callback Method**

Back to Switch States and Execute Actions

Configure the Source that Produces the Parts [Drop-down List]

Our *Source* with the label *Parts In* produces the parts in a **cyclical sequence** according to the settings, which we entered into the *PartsTable*.



Compare the topic **Produce Parts in a Fixed Sequence One Time Only** on how to work with the production table.

Enter the attribute, which sets the *tray* to which the part is going to be moved, into the subtable that double-clicking into the respective cell in the column **Attributes** of the production table opens.

We entered the attribute `Destination` and the respective `Tray`.

.Models.UserControls.PartsTable

.MUs.Board

	object 1	integer 2	string 3	table 4
string	MU	Number	Name	Attributes
1	.MUs.Board	1	T1	t
2	.MUs.Board	1	T2	t
3	.MUs.Board	1	T3	t
4	.MUs.Board	1	T4	t
5				

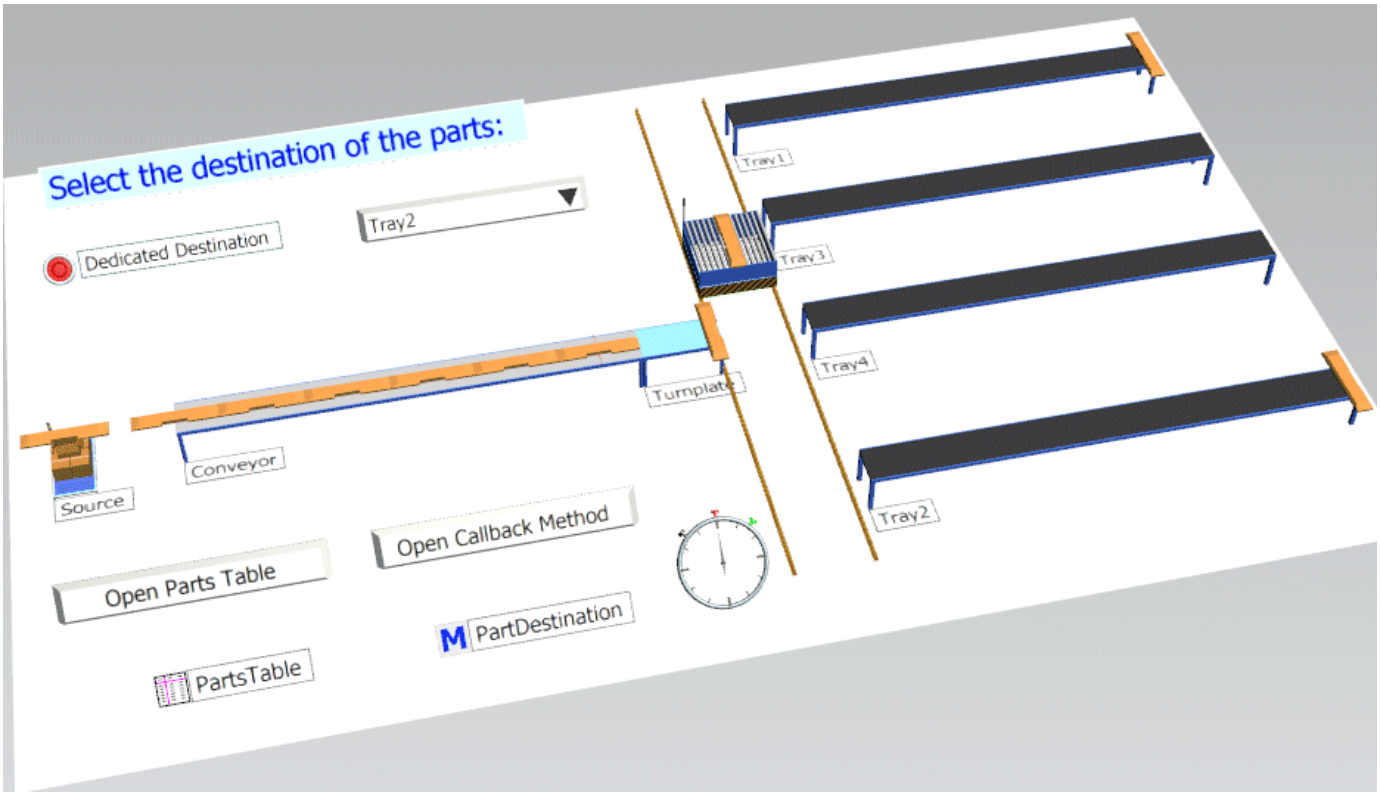
.Models.UserControls.PartsTable[4,1]

Destination

	string 1	integer 2	boolean 3	string 4	rea 5
string	Name of Attribute				
1	Destination			Tray1	
2					
3					
4					
5					

If 

is off, the *Source* produces the part *Board* in a **cyclical sequence** and moves the boards across the materials handling equipment to the destination trays.

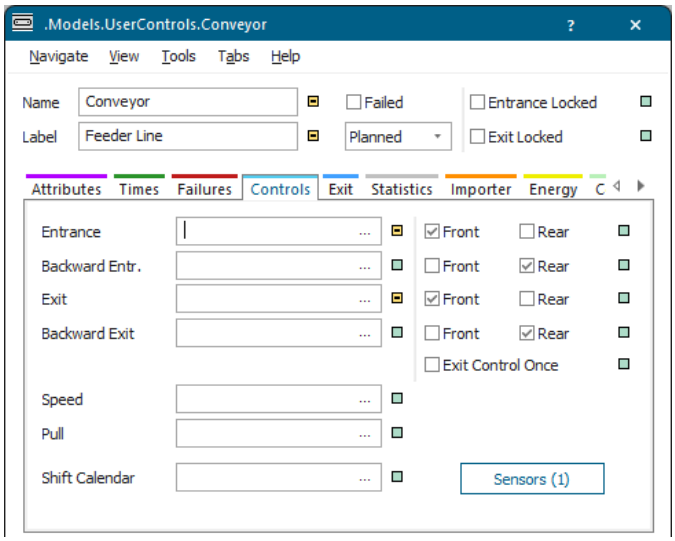


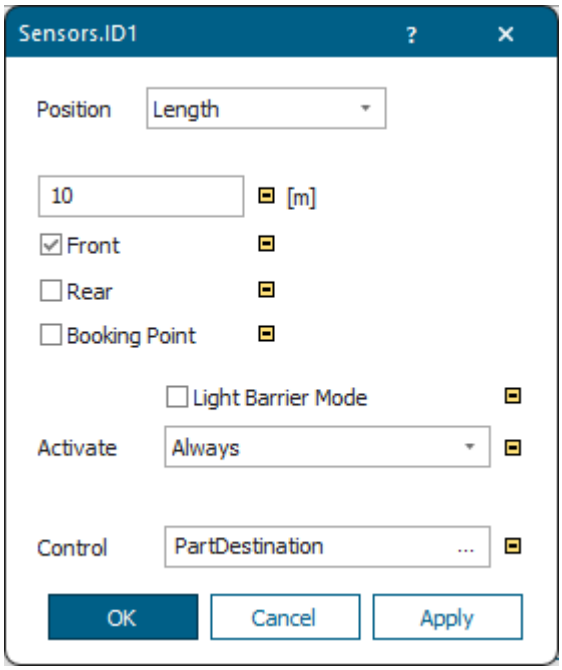
For moving the part, the **Callback Method**, which we enter as the **Sensor Control** into the *feeder line*, uses the attribute *Destination* of the part.

Back to Select an Option from a Drop-down List

Configure the Feeder Line with Sensor and Sensor Control

The *feeder line* is part of the materials handling equipment that transports the parts to their destination trays. Our line is **11** meters long. We created a sensor at **10** meters and entered the method *PartDestination*, which has the label *Callback*, as the **Sensor Control**.





If the check box 

is on, we access the *drop-down list* 

within the callback method *PartDestination* with the attribute **Items**. It then moves the board on to the tray according to the value, which you selected in the *drop-down list*.

The source code looks like this:

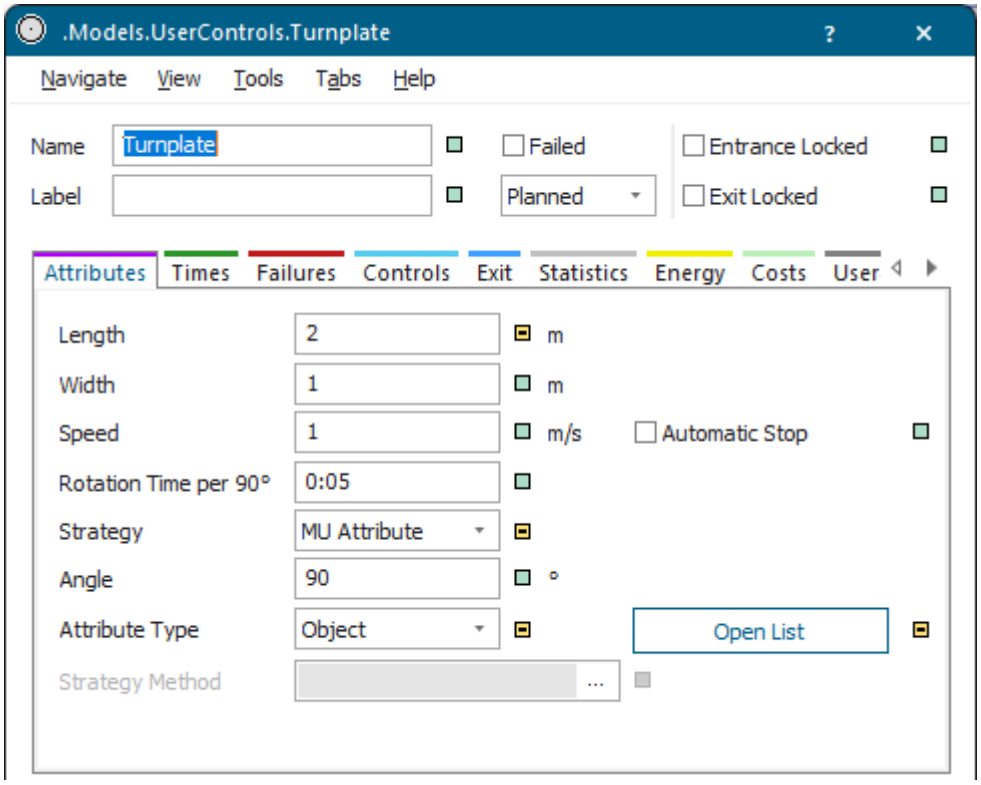
```
// Apply the destination selected in the drop-down list
// named/labeled 'SelectDestination/Dedicated Destination'.
if DestinationActive.Value // destination is active
// set new destination of the part
   @.Destination := SelectDestination.Items[SelectDestination.Value]
end
```

Back to Select an Option from a Drop-down List

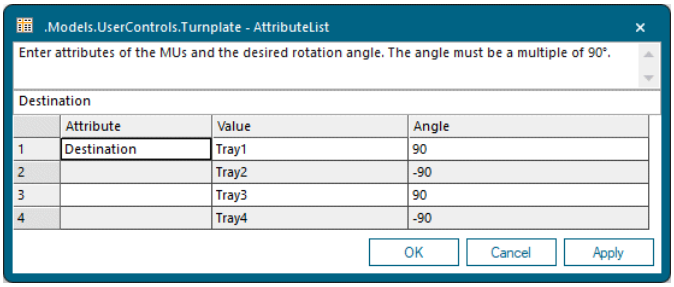
Configure the Turnplate to Rotate the Part According to an Attribute

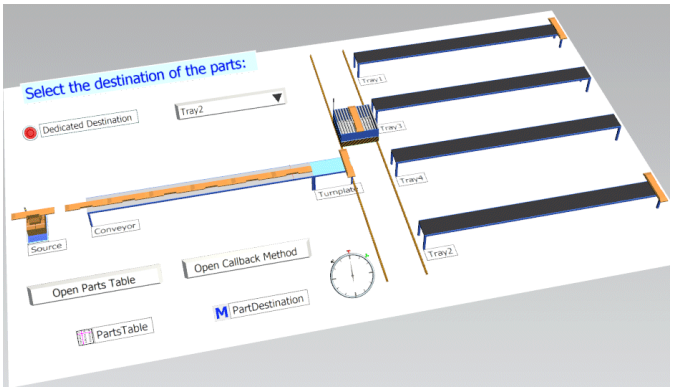
Following the *feeder line* in our plant we added a **Turnplate** to rotate the produced boards.

To do so, we selected the **Strategy > MU Attribute** and the **Attribute Type > Object**, and entered the required values into the **Attribute List**.



We entered the **Attribute** named `Destination`, the names of the destination trays, `Tray1` to `Tray4`, and the **Angle** by which the board will be rotated. 90 stands for 90 degrees clockwise, -90 stands for 90 degrees counterclockwise.





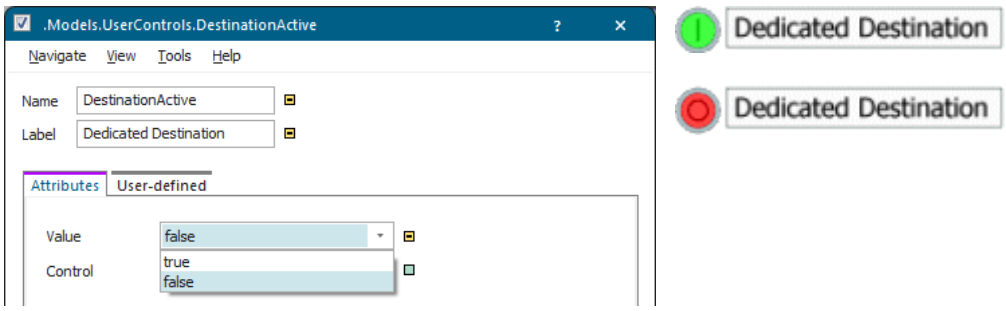
Back to Select an Option from a Drop-down List

Configure the Check Box and the Drop-down List

In our example the *check box* and the *drop-down list* work together:

- The *check box* 

activates or deactivates moving the boards on to a dedicated tray.

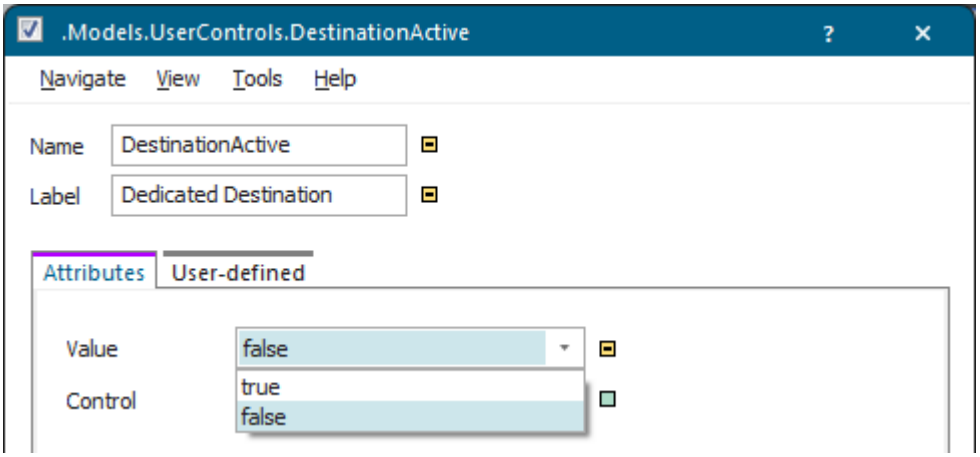


- The *drop-down list* 

sets the dedicated destination of the boards once you click the *check box* to set it to on.

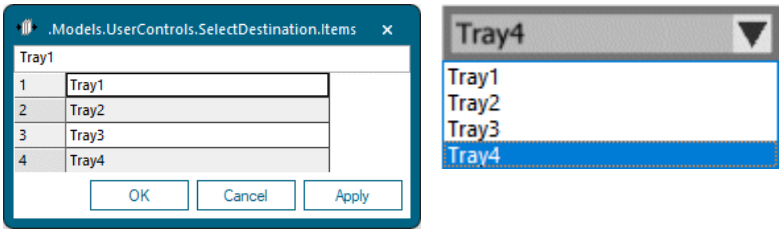
  To configure the *drop-down list*, proceed as follows:

  - Enter the **Width** and the **Height** with which the *drop-down list* will be shown in the *Frame*. We entered a **Width** of 6 meters and the **Height** of 1 meter.



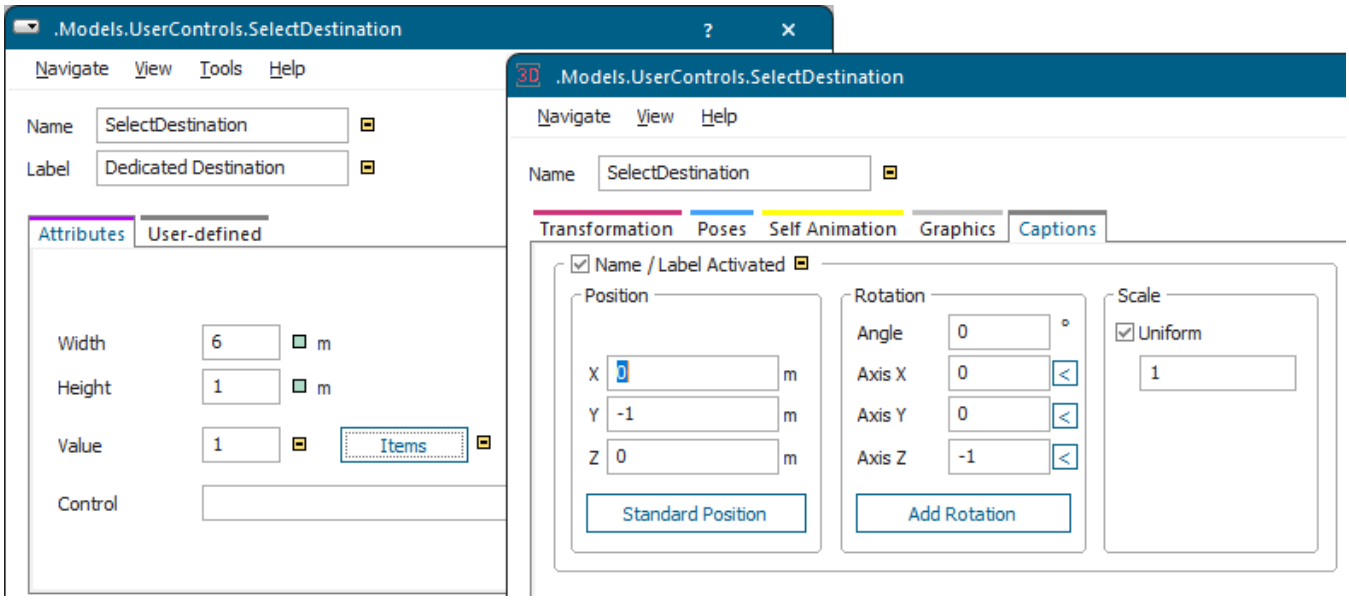
  - Click the button **Items** and enter the items which the *drop-down list* shows in the *Frame*.

    The drop-down list looks like this.

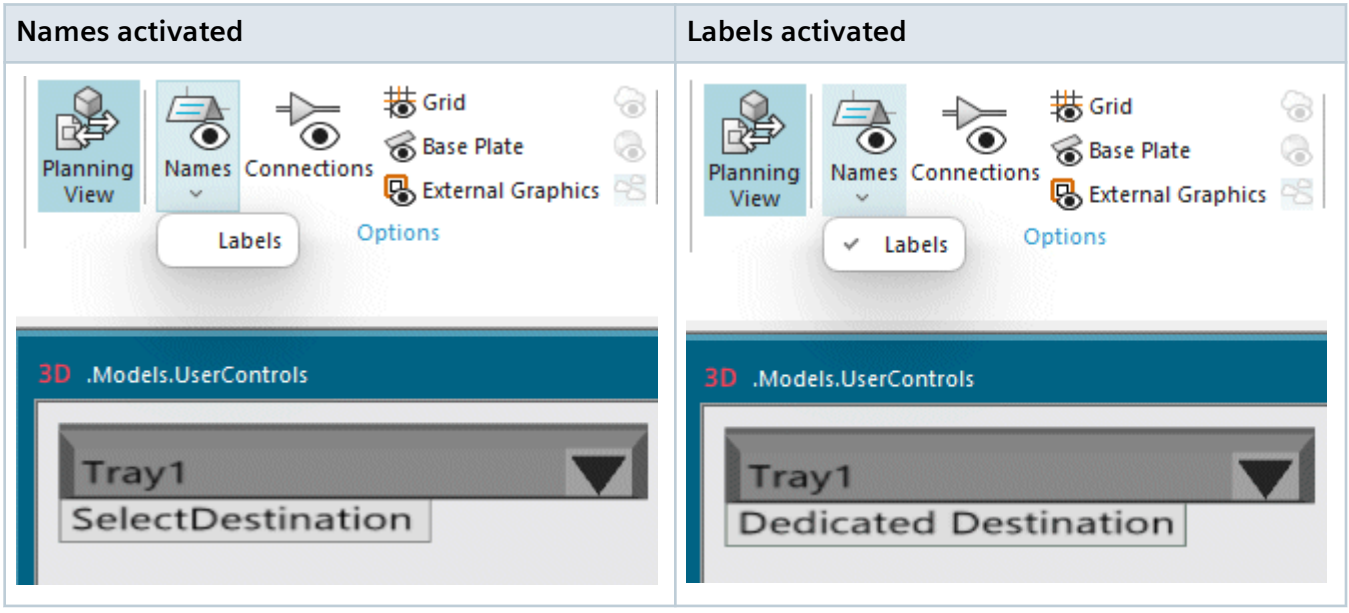


  - Enter the number of the item in the list which the *drop-down list* will show in the *Frame* by default into the text box **Value**. We entered 1 to make the *drop-down list* show **Tray1** when it is closed.

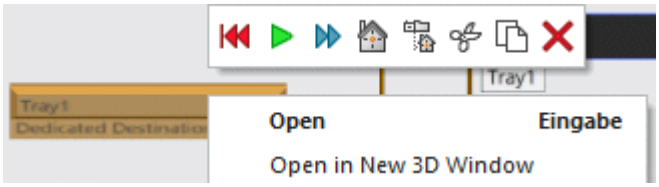
- You can set the caption of the *drop-down list*, show it in the *Frame*, and set its position.



On the **View** ribbon tab you can select if you want to show the **name**, or the **label**, or both in the *Frame*.



To open the dialog of the *drop-down list*, click it with the right mouse button and select **Open** on the context menu.



Note

Plant Simulation only accepts the click the *Checkbox* if it occupies at least 10 pixels in each direction in the model window. If you zoom out very far, the *Checkbox* might become too small, preventing *Plant Simulation* to recognize it reliably.

If the check box 

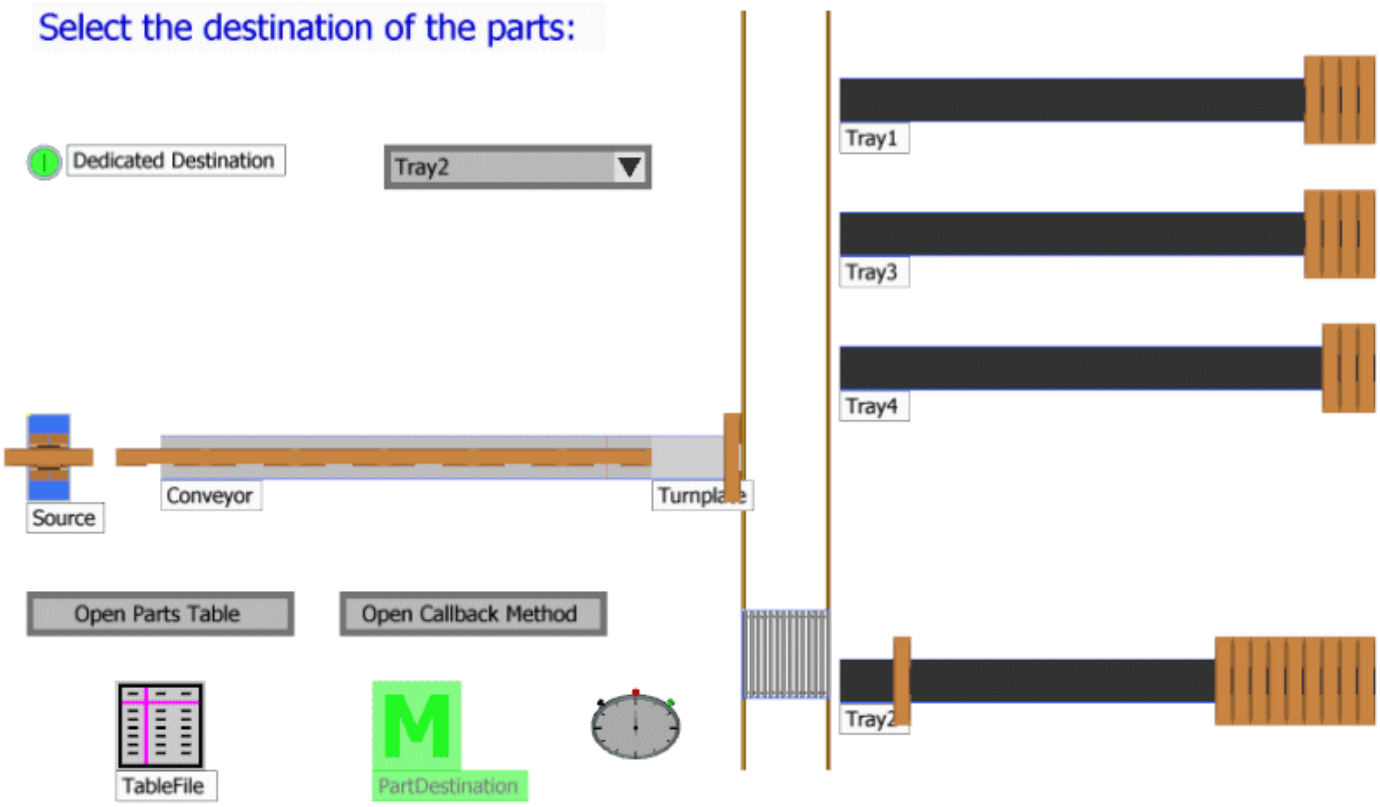
is on, we access the *drop-down list* 

within the **Callback Method** of the sensor with the attribute **Items**. It then moves the board on to the tray according to the value that you selected.

The source code looks like this:

```
// Apply the destination selected in the drop-down list
// named/labeled 'SelectDestination/Dedicated Destination'
var l: list
l.create
SelectDestination.getItems(l)

if DestinationActive.Value then  // destination is active
// set new destination of the part
   @.Destination := l[SelectDestination.Value]
end
```

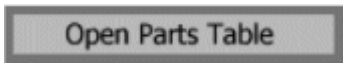


Back to Select an Option from a Drop-down List

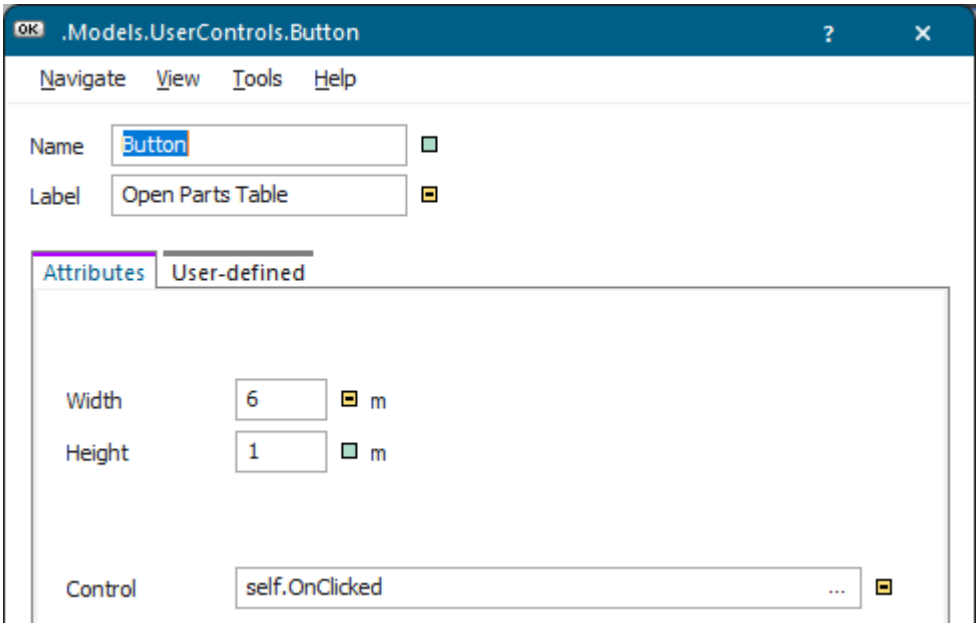
Configure the Buttons to Open Parts Table and Callback Method

We finally thought it to be helpful if we could open the parts table and the callback method with one click instead of having to navigate to them in the *Frame* and double-clicking their icon.

To do so, we configured two buttons:

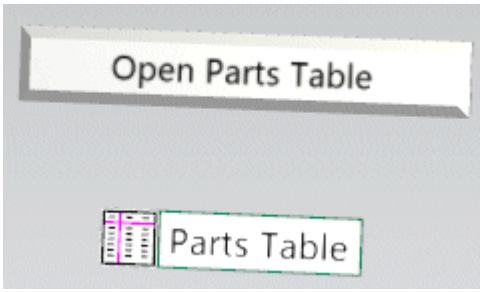
- The *button* 

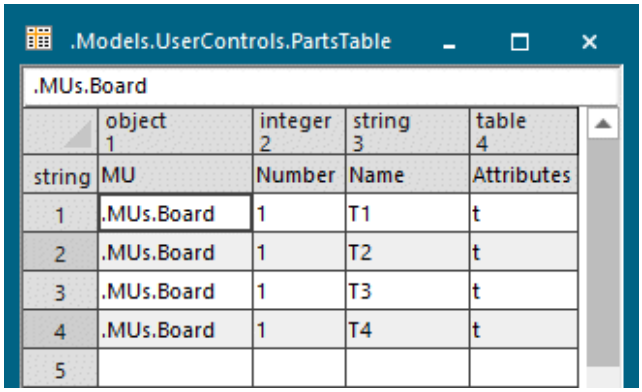
opens the table according to which the *Source* named *PartsIn* produces the parts.
  - Enter the **Width** and the **Height** with which the *button* will be shown in the *Frame*. We entered a **Width** of 6 meters and a **Height** of 1 meter.
  - Enter the **control**, which will be called, when you click the *button*. We decided to use a control that is part of the object. So we right-clicked into the text box **Control** and select **Create Control**.



We then entered the following source code into the control that opened. It opens the parts table named *PartsTable* in the *Frame*.

```
self.~.~.PartsTable.openDialog
```





- The *button* 

opens the **sensor control** of the feeder line that sends the parts on to their destinations according to the settings of the check box 

and of the *drop-down list* 

.
  - Enter the **Width** and the **Height** with which the *button* will be shown in the *Frame*. We entered a **Width** of 6 meters and a **Height** of 1 meter.
  - Enter the control, which will be called, when you click the *button*. We decided to use a control that is part of the object. So we right-clicked into the text box **Control** and select **Create Control**.